Лабораторная работа №3. Секреты в коде: Gitleaks и pre-commit

Тема: поиск секретов в файлах и в истории Git, реагирование на утечку, автоматическая блокировка опасных коммитов. **Время:** 2 академических часа. **Результат:** секрет убран из кода, в репозитории работает проверка перед каждым коммитом, историческая находка оформлена как документированное исключение.

Перед началом: выполнена ЛР2, установлен и запущен Docker.

---

1. Цель работы

Научиться находить секреты в репозитории, правильно на них реагировать и не допускать появления новых.

После работы вы сможете:

- объяснить, почему удаление секрета из файла не устраняет утечку;
- запускать Gitleaks по файлам и по истории Git и читать его отчёт;
- вынести секрет из кода в переменные окружения;
- настроить проверку, которая не даёт создать коммит с секретом;
- оформить исключение так, чтобы оно не превратилось в способ прятать проблемы.

2. Теория

2.1. Что такое секрет

Секрет — любая строка, которая сама по себе даёт доступ: пароль, токен API, закрытый ключ, строка подключения к базе с паролем. Кто знает секрет, тот действует от имени его владельца.

Секреты попадают в код по понятным причинам: «временно вписал, чтобы проверить», «это же закрытый репозиторий», «потом уберу». Автоматические программы постоянно просматривают публичные репозитории, и опубликованный токен находят за минуты.

В нашем проекте в `backend/config.py` лежит строка, похожая на токен GitHub. **Она фиктивная** и никуда доступа не даёт. Работать мы с ней будем как с настоящей.

2.2. Почему важна история Git

Git хранит не только текущее состояние файлов, но и все прошлые. Если вы удалите токен и сделаете новый коммит, старый коммит с токеном останется в истории. Любой, у кого есть копия репозитория, достанет его одной командой.

Отсюда главное правило реагирования:

> Секрет, попавший в репозиторий, считается раскрытым. Его нужно **отозвать и выпустить новый**. Удаление из кода — только уборка после этого.

Порядок действий при настоящей утечке:

1. **Отозвать** секрет у того, кто его выдал (сменить пароль, удалить токен) — с этого момента украденная копия бесполезна.
2. **Выпустить новый** секрет и передать его приложению безопасным способом.
3. **Проверить журналы**: пользовался ли кто-то секретом, пока он был открыт.
4. **Убрать** секрет из кода.
5. **Выяснить причину** и закрыть её: поставить автоматическую проверку.

Переписывание истории Git (удаление секрета из старых коммитов) возможно, но не заменяет отзыва: вы не знаете, кто уже успел сделать копию.

2.3. Где хранить секреты

Код и секреты должны жить отдельно. Простейший способ — **переменные окружения**: значение задаётся там, где запускается программа, а в коде остаётся только имя переменной.

```
import os
TOKEN = os.getenv("DEMO_GITHUB_TOKEN", "")
```

В промышленных системах используют специальные хранилища секретов, а в конвейерах CI — защищённые переменные (в GitHub они называются Secrets). С ними вы встретитесь в ЛР5.

2.4. Как работает Gitleaks

Gitleaks ищет секреты двумя способами:

- **правила для известных форматов** — регулярные выражения для ключей AWS, токенов GitHub, Slack и сотен других сервисов;
- **энтропия** — мера случайности строки. У слова `password` энтропия низкая, у `8ae31cacf141669d` — высокая. Длинная случайная строка рядом со словом `token` или `key` подозрительна, даже если формат неизвестен. Этот способ реализован правилом `generic-api-key`.

Два режима работы:

Команда	Что проверяет	Когда нужна
`gitleaks dir`	файлы в каталоге в их текущем виде	быстрая проверка рабочей копии
`gitleaks git`	все коммиты в истории	проверка репозитория целиком, CI

Параметр `--redact` скрывает найденное значение в выводе. Он нужен, чтобы секрет не утёк ещё раз — уже через отчёт сканера или журнал CI.

Каждая находка получает **отпечаток (fingerprint)** — строку вида `коммит:файл:правило:строка`, которая однозначно её идентифицирует.

2.5. Исключения

Сканер ошибается: тестовые данные и примеры из документации выглядят как секреты. Для таких случаев есть файл `.gitleaksignore` — список отпечатков находок, которые разобраны и признаны неопасными.

Правила работы с исключениями:

- исключение — это **решение человека**, у него есть автор, причина и дата;
- исключается конкретная находка по отпечатку, а не правило целиком и не весь файл;
- настоящий действующий секрет в исключения не вносят никогда — его отзывают.

2.6. Что такое pre-commit

Git умеет запускать программы в определённые моменты — например, перед созданием коммита. Такие программы называют **хуками (hooks)**. Если хук завершился с ошибкой, коммит не создаётся.

`pre-commit` — менеджер хуков. В файле `.pre-commit-config.yaml` перечислены проверки, а команда `pre-commit install` подключает их к вашему репозиторию.

Ограничение: хук работает на компьютере разработчика, и его можно обойти (`git commit --no-verify`) или просто не установить. Поэтому локальная проверка — это **удобство**: быстрая обратная связь до того, как секрет покинул ноутбук. **Гарантию** даёт только та же проверка на сервере, в конвейере CI — её вы настроите в ЛР5.

3. Ход работы

Все команды выполняются в каталоге `~/devsecops-lab` с активированным окружением.

```
cd ~/devsecops-lab
source .venv/Scripts/activate      # Windows, Git Bash
# source .venv/bin/activate        # Linux и macOS
```

Только для Windows (Git Bash). Выполните в каждом новом окне терминала перед командами `docker run`:

```
export MSYS_NO_PATHCONV=1
```

Без этого Git Bash превращает путь `/src` внутри контейнера в путь Windows, и сканер не находит файлы.

Шаг 1. Проверить Docker и получить Gitleaks

```
docker run --rm ghcr.io/gitleaks/gitleaks:v8.24.2 version
```

Ожидаемый результат: `8.24.2`. При первом запуске Docker скачает образ — это займёт около минуты.

Команда запуска длинная, поэтому разберём её один раз:

- `docker run --rm` — запустить контейнер и удалить его после завершения;
- `-v "$PWD:/src"` — показать контейнеру текущий каталог под именем `/src`;
- `ghcr.io/gitleaks/gitleaks:v8.24.2` — образ с Gitleaks; версия указана явно, чтобы результат был воспроизводимым.

Если не получилось: `Cannot connect to the Docker daemon` — не запущен Docker Desktop.

Шаг 2. Найти секрет в файлах

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 dir --redact -v
 /src
echo "Код завершения: $?"
```

Ожидаемый результат:

```
Finding:     DEMO_GITHUB_TOKEN = "REDACTED"
Secret:      REDACTED
RuleID:      generic-api-key
Entropy:     4.020844
File:        /src/backend/config.py
Line:        3
Fingerprint: /src/backend/config.py:generic-api-key:3

WRN leaks found: 1
Код завершения: 1
```

Код завершения 1 означает «найдены утечки». Это штатный результат, а не сбой программы.

Обратите внимание: сработало правило `generic-api-key`, а не правило для токенов GitHub. Настоящий токен GitHub длиннее нашего учебного, поэтому точное правило его не узнало — выручила проверка энтропии.

В отчёт: вывод команды. Значение секрета в отчёт не копируйте.

Шаг 3. Найти секрет в истории

```
mkdir -p reports
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 --report-format json --report-path /src/reports/gitleaks-before.json /src
```

Ожидаемый результат: та же находка, но с дополнительными полями `Commit`, `Author`, `Date`. Отпечаток теперь начинается с идентификатора коммита.

Заполните карточку находки:

Поле	Значение
Правило (RuleID)	
Файл и строка	

Поле	Значение
Коммит (первые 7 символов)	
Автор и дата коммита	
Отпечаток (Fingerprint)	
Что это за секрет и к чему он дал бы доступ, будь он настоящим	

В отчёт: карточка находки.

Шаг 4. Убрать секрет из кода

Откройте `backend/config.py` и замените его содержимое на:

```
"""Настройки учебного приложения. Секреты приходят из переменных окружения."""
import os

DEMO_GITHUB_TOKEN = os.getenv("DEMO_GITHUB_TOKEN", "")
FLASK_SECRET_KEY = os.getenv("FLASK_SECRET_KEY", "dev-only-change-me")
DATABASE_PATH = "data/students.db"
```

Проверьте, что приложение не сломалось и что файлы теперь чистые:

```
pytest -q
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 dir --redact -v
 /src
```

Ожидаемый результат: `7 passed`, затем `no leaks found`.

Сохраните исправление:

```
git add -A
git commit -m "lab3: read secrets from environment"
```

> У ключа `FLASK_SECRET_KEY` осталось значение по умолчанию `dev-only-change-me`. Gitleaks о нём молчит: строка не похожа на случайную. Но это тоже проблема — запишите её в остаточные риски. Подумайте, чем опасен общеизвестный ключ подписи сессий.

Если `gitleaks dir` нашёл что-то в папке `reports` — это отчёты других сканеров, в которых встречаются длинные случайные строки. В коммиты эта папка не попадает. Ложные срабатывания в `reports` в этой работе можно не учитывать.

Шаг 5. Убедиться, что история помнит всё

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src
```

Ожидаемый результат: `leaks found: 1`. Файл чист, а секрет по-прежнему доступен в первом коммите. Убедитесь сами:

```
git show HEAD~2:backend/config.py
```

Команда печатает файл в том виде, в каком он был два коммита назад, — вместе с токеном.

Опишите в отчёте, что вы сделали бы, будь этот токен настоящим. Используйте пять шагов из раздела 2.2 и укажите для каждого, **кто** его выполняет и **где** (например: «отзыв — владелец учётной записи GitHub, в настройках Developer settings → Personal access tokens»).

В отчёт: вывод двух команд и план реагирования.

Шаг 6. Установить проверку перед коммитом

Откройте `.pre-commit-config.yaml`:

```yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.24.2
    hooks:
      - id: gitleaks
```

Здесь `repo` — откуда взять проверку, `rev` — её точная версия, `id` — какую именно проверку запускать.

Установите хук и проверьте им все файлы:

```
pre-commit install
pre-commit run --all-files
```

Ожидаемый результат:

```
pre-commit installed at .git/hooks/pre-commit
Detect hardcoded secrets.....................................................Passed
```

Первый запуск занимает несколько минут: pre-commit скачивает и собирает Gitleaks. Следующие запуски — доли секунды.

Шаг 7. Попробовать закоммитить секрет

Проверка, которую ни разу не видели в действии, ничего не гарантирует. Создайте файл с «секретом» и попробуйте его сохранить:

```
printf 'token=ghp_8ae31cacf141669ddfb5da7e31c8a2f3a4\n' > secret-demo.txt
git add secret-demo.txt
git commit -m "Test secret blocking"
```

Ожидаемый результат:

```
Detect hardcoded secrets.....................................................Failed
- hook id: gitleaks
- exit code: 1
...
File:        secret-demo.txt
Line:        1
```

Убедитесь, что коммит не создан, и уберите файл:

```
git log --oneline
git restore --staged secret-demo.txt
rm secret-demo.txt
git status
```

Ожидаемый результат: в истории нет коммита `Test secret blocking`, рабочая копия чистая.

В отчёт: вывод неудавшегося `git commit` и вывод `git log --oneline` после него.

Шаг 8. Оформить исключение для исторической находки

Сейчас `gitleaks git` всегда завершается с ошибкой из-за первого коммита. В ЛР5 эта же команда будет работать в конвейере, и он всегда будет «красным». Проверка, которая горит постоянно, бесполезна: на неё перестают смотреть и пропускают новые находки.

Находка разобрана: токен учебный, из текущего кода удалён. Это основание оформить исключение.

Получите отпечаток находки:

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src | grep Fingerprint
```

Создайте в корне проекта файл `.gitleaksignore`. В первой строке — комментарий с обоснованием, во второй — отпечаток **из вашего вывода** (идентификатор коммита у каждого студента свой):

```
# Учебный токен из первого коммита: фиктивный, из текущего кода удалён в ЛР3. <Ф
амилия>, <дата>
162fb607f51d8fda36ca6ba640f692d4eb9bac0c:backend/config.py:generic-api-key:3
```

Проверьте и сохраните:

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src
git add -A
git commit -m "lab3: document historical finding in .gitleaksignore"
```

Ожидаемый результат: `no leaks found`; коммит создан, хук при этом показал `Passed`.

Если по-прежнему `leaks found: 1` — отпечаток скопирован неточно. Он должен совпадать с выводом Gitleaks до символа, без пробелов в начале и в конце.

В отчёт: содержимое `.gitleaksignore` и ответ на вопрос: чем это исключение отличается от попытки спрятать настоящую утечку?

Шаг 9. Итоговая проверка

```
semgrep scan --config .semgrep.yml --metrics=off --error .
git log --oneline
```

Ожидаемый результат: Semgrep сообщает `0 findings` — последняя находка из ЛР2 (`lab.github-token-literal`) устранена. В истории четыре коммита.

4. Требования к отчёту

1. Вывод `gitleaks dir` и `gitleaks git` для исходного состояния.
2. Карточка находки (без значения секрета).
3. Новое содержимое `config.py` и результаты проверки после исправления.
4. Доказательство, что секрет остался в истории, и план реагирования из пяти шагов.
5. Доказательство работы хука: вывод заблокированного коммита.
6. Файл `.gitleaksignore` с обоснованием.
7. Остаточные риски: что Gitleaks не нашёл и почему.
8. Вывод.

5. Контрольные вопросы

1. Почему удаление токена новым коммитом не устраняет утечку?
2. Чем режим `dir` отличается от режима `git`?
3. Зачем нужен параметр `--redact`? Где ещё может «утечь» секрет, кроме кода?
4. Что такое энтропия строки и как она помогает искать секреты?
5. Почему локального хука недостаточно и проверку повторяют в CI?
6. В каком случае находку можно внести в `.gitleaksignore`, а в каком — нельзя?
7. Почему в `.pre-commit-config.yaml` указана точная версия `rev`, а не «последняя»?

6. Критерии оценивания

Критерий	Баллы
Сканирование файлов и истории, карточка находки	20
Вынос секрета в переменные окружения, тесты проходят	20
План реагирования на утечку	15
Установка pre-commit и доказательство блокировки	25
Обоснованное исключение, остаточные риски, выводы	20
Итого	**100**

7. Источники

- ГОСТ Р 56939-2024. Разработка безопасного программного обеспечения. Общие требования.
- Gitleaks: https://github.com/gitleaks/gitleaks
- pre-commit: https://pre-commit.com/
- OWASP Secrets Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html